



Master in Computer Science
Software Engineering

Cloud Computing

Report 2

Healthcare Management System Architecture

Team Members

Jordy Meye
Huizhe Su
Arihant Jain
Nizar

March 2026

Contents

1	System Architecture Overview	2
1.1	Objective	2
2	Architecture Design	3
2.1	Visual Representation	3
3	Detailed Components	4
3.1	Interoperability	4
3.2	Telemedicine Setup	4
3.3	Scalability and Performance	4
3.4	Data Flow Example	4
3.5	Disaster Recovery (DR)	4
4	Cost Optimization	5
5	AI Integration	5
6	System Monitoring	5
7	Reflection on Proposed Architecture	6
7.1	Scalability Impact	6
7.2	User Accessibility	6
7.3	Interoperability Challenges	6
7.4	Telemedicine Ethical Considerations	6
7.5	Data Security Measures	6
7.6	Resource Optimization	6
7.7	Adaptability to Technological Advancements	6
7.8	User Training and Support	6
7.9	AI Decision Transparency	6
7.10	Disaster Recovery and Contingency Planning	6
8	Conclusion	7

Exercise 1. System Architecture Overview

Question 1. Objective

In this lab, we designed a cloud-based Healthcare Management System on AWS. The goal was to build a scalable and secure architecture that can handle patient data, telemedicine, and integrations with existing hospital systems.

The lab required us to cover these areas:

- Interoperability with existing hospital systems (HL7/FHIR).
- A telemedicine infrastructure for video consultations.
- Multi-platform user access.
- AI and analytics for clinical decision support.
- High scalability and performance.
- Cost control.
- Security, HIPAA compliance, and disaster recovery.

Exercise 2. Architecture Design

Question 1. Visual Representation

We drew an architecture diagram to map out all the components and how data moves between them. The system is split into layers to keep the frontend, backend, data storage, and security concerns separate.

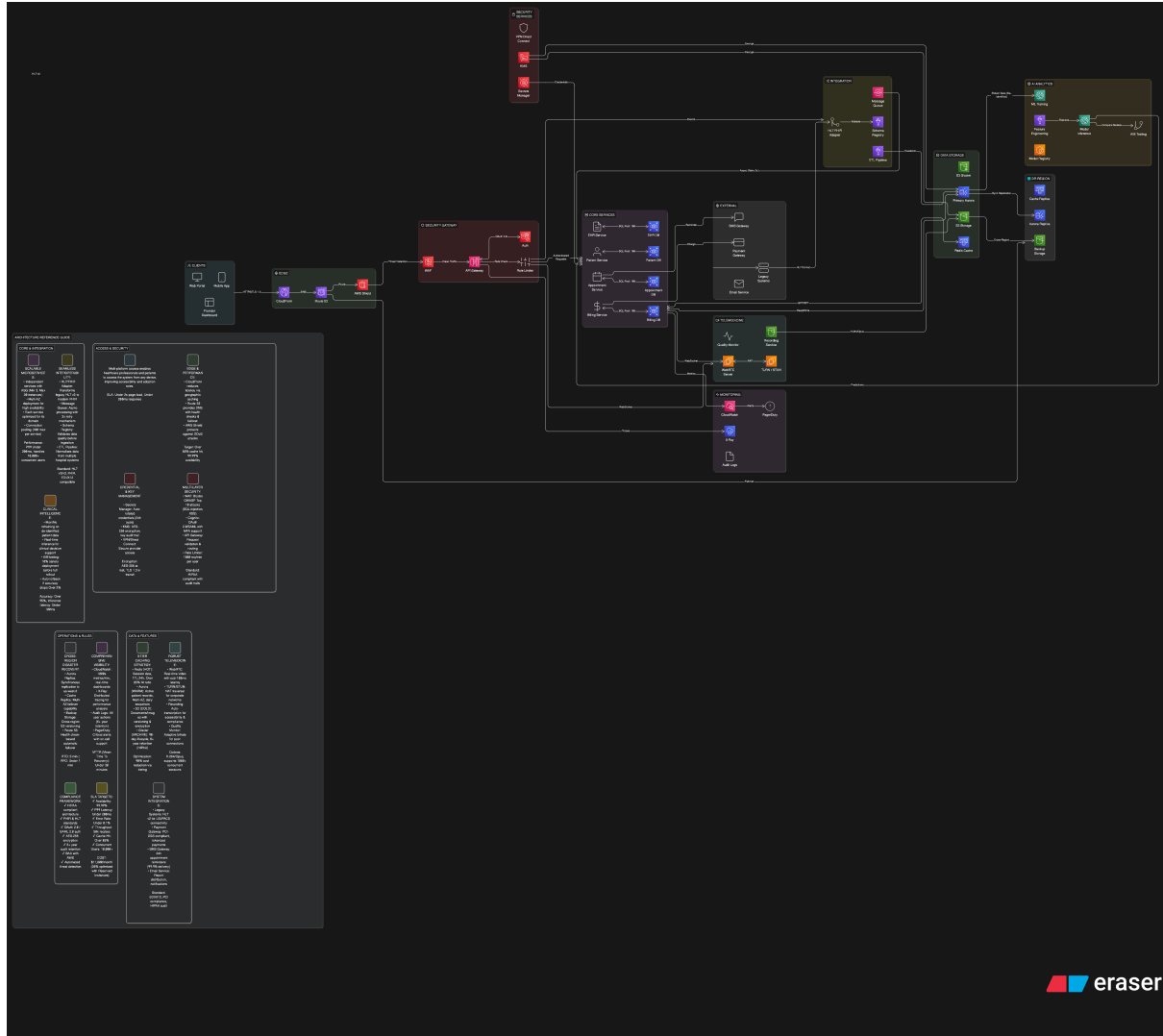


Figure 1: Healthcare Management System Architecture

The main layers in our design are:

- **Client Layer:** Web portals and mobile apps for patients and providers.
- **Edge & Security:** CloudFront for caching and AWS WAF to filter incoming web traffic.
- **Core Services:** EC2 instances running microservices for health records, billing, and appointments.
- **Data Storage:** Redis for caching, Aurora for the main database, and S3 for file storage.
- **AI & Analytics:** SageMaker for machine learning models.
- **Disaster Recovery:** Cross-region replication to a secondary AWS region.

Exercise 3. Detailed Components

Question 1. Interoperability

To connect with legacy hospital software, we added an HL7/FHIR adapter at the entry point. Incoming data goes through an AWS Glue schema registry for validation. We also added an SQS queue to buffer messages in case the backend is under load, so nothing gets dropped during a spike.

Question 2. Telemedicine Setup

For video consultations, we set up a WebRTC server on EC2. We included TURN/STUN servers to handle users behind corporate firewalls. Calls are automatically recorded, the audio is extracted, and the files are saved to S3.

Question 3. Scalability and Performance

All core microservices are inside Auto Scaling Groups (ASG).

- The ASG scales between 3 and 20 instances based on CPU utilization.
- An Application Load Balancer distributes the traffic across instances.
- We use ElastiCache (Redis) to cache frequent queries and keep API response times under 200ms without hitting the Aurora database every time.

Question 4. Data Flow Example

When a patient registers on the web app, the request goes through the API Gateway, gets authenticated by Cognito, and is saved to the Aurora Patient DB. When they book a telemedicine call, the Appointment Service schedules it and triggers the SMS Gateway to send a reminder. During the call, the WebRTC server handles the video connection. Afterward, the recording goes to S3, and the anonymized text data is pushed to SageMaker to help retrain the clinical AI models.

Question 5. Disaster Recovery (DR)

We set up the primary region to continuously replicate data to a standby region.

- Aurora uses cross-region replication to keep the backup database in sync.
- Route 53 health checks monitor the primary region. If it goes down, DNS automatically fails over to the backup.
- Our target RTO is 5 minutes.

Exercise 4. Cost Optimization

To keep AWS costs reasonable, we used a few different strategies:

1. **Reserved Instances:** Databases and base EC2 servers run 24/7, so we plan to use Reserved Instances for them. It gives a significant discount compared to On-Demand pricing.
2. **Data Tiering:** We added S3 lifecycle rules. Patient documents and old recordings stay in S3 Standard for 90 days, then get moved to S3 Glacier automatically.
3. **Spot Instances:** Monthly ML retraining jobs use Spot Instances since they can tolerate interruptions and don't need to finish immediately.
4. **CDN Caching:** Serving static assets through CloudFront reduces data transfer costs from the origin servers.

Exercise 5. AI Integration

We used AWS SageMaker to give doctors clinical decision support. The models can predict disease risks and flag potential drug interactions.

To keep the models accurate, they are retrained monthly using de-identified patient data. When we deploy a new model, we use A/B testing and send only 10% of traffic to the new version first. If accuracy drops or error rates increase, the system rolls back automatically to the previous model.

Exercise 6. System Monitoring

We set up several monitoring tools to track the health of the system:

- **CloudWatch:** Collects metrics for CPU, memory, and network. We configured alarms to trigger when availability drops below 99.99%.
- **X-Ray:** Traces requests across the microservices to help find bottlenecks.
- **Audit Logs:** All user actions are logged and stored in S3 for HIPAA compliance.
- **PagerDuty:** Integrated with CloudWatch to alert the on-call team if a critical service goes down.

Exercise 7. Reflection on Proposed Architecture

Question 1. Scalability Impact

Using Auto Scaling Groups and a Load Balancer means the system can handle sudden traffic spikes. One thing we thought about was database connection limits during rapid growth. We handled this by adding connection pooling and using Redis to cache reads so not everything goes straight to Aurora.

Question 2. User Accessibility

We kept the UI flat and simple for users who aren't very technical. The mobile app also includes a voice-to-text feature so patients can describe symptoms without typing. We also added high-contrast mode and large text options.

Question 3. Interoperability Challenges

The main issue with legacy hospital systems is that HL7 messages can be outdated or badly formatted. We put an AWS Glue schema registry in front of the database to catch these. If a message is invalid, it goes into a dead-letter queue for manual review instead of crashing the pipeline.

Question 4. Telemedicine Ethical Considerations

All video traffic is encrypted with WebRTC. Recordings are access-controlled and deleted after 90 days. If a patient's connection becomes unstable during a call, the system switches to audio-only automatically to avoid dropping the session entirely.

Question 5. Data Security Measures

We layered security across the whole stack. AWS WAF sits at the edge to block common exploits. The API Gateway handles rate limiting, and Cognito enforces MFA for all staff accounts. Data is encrypted at rest using KMS (AES-256) and in transit with TLS 1.3.

Question 6. Resource Optimization

We balanced cost and performance by using S3 lifecycle policies to move old records to Glacier, Reserved Instances for steady workloads, and Spot Instances for background ML training. We will use AWS Cost Explorer to track billing over time.

Question 7. Adaptability to Technological Advancements

Since we used a microservices architecture, we can update or swap out individual components without touching the rest of the system. For example, adding an IoT patient monitoring service would only require deploying a new microservice. Using the FHIR standard also makes it easier to integrate with future medical APIs.

Question 8. User Training and Support

The provider dashboard has built-in tooltips. The patient portal has a help section with simple step-by-step guides. We also planned for a ticketing system so users can contact IT support for login or navigation issues.

Question 9. AI Decision Transparency

The interface shows doctors a confidence score and a reason for each AI prediction. To reduce bias, we made sure the training data includes diverse demographics and we test models against different groups before pushing to production.

Question 10. Disaster Recovery and Contingency Planning

We rely on a Multi-AZ deployment with a secondary AWS region as the backup. Aurora continuously syncs to the backup region. If the primary region goes down, Route 53 health checks trigger an automatic failover. We also run periodic drills to confirm the recovery time stays under 5 minutes.

Exercise 8. Conclusion

In this lab, we designed a cloud architecture for a Healthcare Management System using AWS. The system covers all the requirements from the lab: interoperability with legacy systems, telemedicine support, HIPAA compliance, and cost optimization through data tiering and instance reservations. Using managed services kept the operational overhead low while still giving us the flexibility to scale.